

CampusX — 100 Days of Machine Learning

Complete Theory Notes · Day 21 to Day 30

Based on the official YouTube playlist by Nitish Singh (CampusX)

Day	Topic	Video ID
21	EDA — Bivariate & Multivariate Analysis	6D3VtEfCw7w
22	Pandas Profiling	E69Lg2ZgOxg
23	What is Feature Engineering?	sluoVhT0ehg
24	Feature Scaling — Standardization	1Yw9sC0PNwY
25	Feature Scaling — Normalization	eBrGyuA2MIg
26	Encoding Categorical Data (Ordinal & Label)	w2GglmYHfmM
27	One Hot Encoding	U5oCv3JKWKA
28	Column Transformer	5TVj6iEBR4I
29	ML Pipelines	xOccYkgRV4Q
30	Function Transformer (Log, Reciprocal, Sqrt)	cTjj3LE8E90

What is Bivariate Analysis?

- **Bivariate Analysis** = studying the relationship between **two variables** simultaneously to find correlations, patterns, and dependencies.
- Goal: understand how one variable changes with respect to another. Helps identify which features are useful predictors for your target variable.

Case 1 — Numerical vs Numerical

- **Scatter Plot**: x-axis = feature1, y-axis = feature2. Shows direction and strength of relationship.
 - Positive correlation: both increase together (height vs weight).
 - Negative correlation: one increases, other decreases (price vs demand).
 - No correlation: random scatter, no clear pattern.
- **Pearson Correlation Coefficient (r)**: Measures linear relationship. Range -1 to +1.
 - $r > 0.7$ = strong positive. $r < -0.7$ = strong negative. $r \approx 0$ = no linear relationship.
 - Only detects LINEAR relationships — two variables can have a strong non-linear relationship but $r \approx 0$.

```
# Scatter plot sns.scatterplot(x='cgpa', y='salary', data=df) # Correlation heatmap (all numerical columns at once) sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
```

Case 2 — Numerical vs Categorical

- **Box Plot grouped by category**: Compare distributions of a numerical feature across categories.
 - Example: salary distribution for each department — shows median, spread, and outliers per group.
- **Bar Plot (mean/median per group)**: Shows average of numerical feature for each category.
- **Violin Plot**: Combines boxplot + KDE — shows full distribution shape per category.

```
sns.boxplot(x='department', y='salary', data=df) # grouped boxplot sns.barplot(x='gender', y='score', data=df) # mean per category sns.violinplot(x='survived', y='age', data=df)
```

Case 3 — Categorical vs Categorical

- **Stacked Bar Chart / Grouped Bar Chart**: Shows frequency of one category within each group of another.
- **Cross-tabulation (pd.crosstab)**: Frequency table of two categorical variables.
- **Heatmap of crosstab**: Visualizes the cross-tab as color intensities — easy to spot patterns.

```
pd.crosstab(df['gender'], df['purchased']) # frequency table sns.countplot(x='department', hue='gender', data=df) # grouped bar
```

Multivariate Analysis

- **Multivariate** = 3+ variables at once. Main tools:
 - **Pair Plot (sns.pairplot)**: Scatter plots for every pair of numerical columns in one grid. Very useful for quick overview of all relationships.
 - **Correlation Heatmap**: Shows pairwise correlations of all numerical features simultaneously.
 - **Color-coded Scatter**: Use hue parameter to add a 3rd categorical dimension to a scatter plot.

```
sns.pairplot(df, hue='target') # all pairs, colored by class sns.heatmap(df.corr(), annot=True) # full correlation matrix
```

Key EDA insight:

Bivariate analysis between each feature and the target column tells you which features are actually useful. A feature with near-zero correlation to the target and no visible pattern in scatter plots is likely a bad predictor — consider dropping it.

What is Pandas Profiling?

- **Pandas Profiling** (now called **ydata-profiling**) is a library that generates a complete, interactive EDA report from a DataFrame in a single line of code.
- It automates everything you'd do manually: univariate stats, distributions, missing values, correlations, duplicate detection, and more.
- Output: a rich HTML report that you can open in any browser or embed in a Jupyter notebook.

What the Report Contains

Section	What it Shows
Overview	Dataset shape, memory size, number of variables, missing cells %, duplicate rows %
Variables	For each column: type, unique values, missing %, histogram/bar chart, descriptive stats
Interactions	Scatter plots for pairs of numerical variables (bivariate)
Correlations	Pearson, Spearman, Kendall, and Cramér's V correlation matrices
Missing Values	Heatmap showing where missing values are — reveals if missing is random or patterned
Sample	First and last 10 rows of the dataset

How to Use

```
# Install pip install ydata-profiling # Generate report from ydata_profiling import ProfileReport profile
= ProfileReport(df, title='My Dataset EDA', explorative=True) # In Jupyter Notebook
profile.to_notebook_iframe() # Save as HTML file profile.to_file('eda_report.html')
```

- **minimal=True** parameter: Generates a faster, lighter report — useful for large datasets (>1M rows).
- **When to use:** At the very start of any new dataset before writing any manual EDA code. Takes 2 seconds to get a comprehensive picture.

Important note:

Pandas Profiling is a quick exploration tool, NOT a replacement for domain-specific EDA. The automated report shows you what to investigate further — it doesn't interpret the results for you. Always follow up on alerts and warnings the report flags (high correlation, high cardinality, etc.).

DAY
23

What is Feature Engineering?

youtu.be/sluoVhT0ehg

Definition

- **Feature Engineering** is the process of using domain knowledge and data transformation techniques to create, modify, or select features (input variables) that make ML models perform better.
- Raw data is rarely in the right form for an ML model. Feature engineering bridges the gap between raw data and model-ready data.
- **Why it matters:** A simple model with excellent features consistently outperforms a complex model with poor features. Feature engineering is often the single highest-ROI activity in an ML project.

Types of Feature Engineering

Type	What it does	Example
Feature Transformation	Change the mathematical scale or distribution of existing features	Log transform of skewed salary column, Standardization, Normalization
Feature Encoding	Convert non-numerical data into numbers the model can use	One Hot Encoding for cities, Ordinal Encoding for ratings (Low/Med/High)
Feature Construction	Create entirely new features by combining or deriving from existing ones	price_per_sqft = price / area, is_weekend from date column, age from birth_year
Feature Splitting	Break one column into multiple more useful columns	Split 'Full Name' → 'First Name' + 'Last Name', split datetime → hour, day, month

Feature Selection	Remove irrelevant, redundant, or noisy features	Drop columns with near-zero variance, drop highly correlated duplicates, use RFE
Feature Scaling	Normalize numerical ranges so algorithms treat features equally	StandardScaler for KNN/SVM, MinMaxScaler for Neural Networks

Why Feature Engineering is Needed

- **Algorithms have assumptions:** Linear Regression assumes linear relationship between features and target. If the real relationship is log-linear, you need to log-transform first.
- **Scale sensitivity:** Distance-based algorithms (KNN, SVM, K-Means) and gradient-based algorithms (Neural Networks, Logistic Regression) are sensitive to feature scale. A feature in thousands will dominate features in single digits.
- **Categorical data:** Most ML algorithms only accept numbers. Categorical columns must be encoded before feeding to sklearn.
- **Missing values:** sklearn models throw errors on NaN. Must be handled before training.
- **Distribution shape:** Many algorithms work better with normally distributed features. Log transform can convert a right-skewed distribution to near-normal.

Feature Engineering vs Feature Selection

- **Feature Engineering:** Creating and transforming features — adding information.
- **Feature Selection:** Choosing which features to keep — removing noise.
- Both are needed. Engineer first (create useful features), then select (keep only the best ones).

The golden rule of Feature Engineering:

A feature is good if it has a strong relationship with the target variable AND provides information not already captured by other features. Domain knowledge is your most powerful tool here — understanding the business/problem context tells you what new features would be meaningful.

DAY
24

Feature Scaling — Standardization

youtu.be/1Yw9sC0PNwY

Why Feature Scaling is Needed

- **Feature Scaling** brings all numerical features onto the same scale so that no single feature dominates others due to its magnitude.
- Without scaling, a feature like Salary (₹50,000) will dwarf features like Age (25) or CGPA (8.5) in distance calculations and gradient updates — the model treats high-magnitude features as more important.
- **Algorithms that NEED scaling:** KNN, SVM, K-Means, Logistic Regression, Neural Networks, PCA, Linear Regression (for gradient descent to converge faster).
- **Algorithms that DON'T need scaling:** Decision Tree, Random Forest, XGBoost, Naive Bayes — they use rules/probabilities, not distances or gradients.

Standardization (Z-score Normalization)

- **Formula:** $z = (x - \text{mean}) / \text{std_deviation}$
- **Output:** Mean = 0, Standard Deviation = 1. The values are centered around zero.
- **Range:** No fixed range — values can go from roughly -3 to +3 for normally distributed data.
- **Effect:** Removes the mean and scales to unit variance. Does NOT make data normally distributed — only rescales it.
- **Robust to outliers?** No — outliers affect the mean and std. If data has outliers, use RobustScaler instead.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train, transform train
X_test_scaled = scaler.transform(X_test) # only transform test (never fit on test)
print(scaler.mean_) # mean used for scaling
print(scaler.scale_) # std used for scaling
```

When to Use Standardization

- Standardization is the **default choice** for most ML tasks. When in doubt, use StandardScaler.
- Works well when data is approximately normally distributed (bell curve shape).
- Preferred over normalization when: data has outliers (less sensitive than MinMax), or algorithm doesn't require bounded [0,1] output.

Critical rule — never fit on test data:

Always call `.fit_transform()` on training data only. For test data, call `.transform()` using the scaler already fitted on training data. Fitting on test data = data leakage — the model 'sees' test statistics during training.

DAY
25

Feature Scaling — Normalization

youtu.be/eBrGyuA2Mlg

Min-Max Scaling (MinMaxScaler)

- **Formula:** $x_{\text{scaled}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$
- **Output range:** `[0, 1]` — all values squeezed between 0 and 1.
- **Effect:** Preserves the original distribution shape. Just changes the scale.
- **Sensitive to outliers:** Yes — one extreme outlier forces all other values to cluster near 0 or 1. A salary of ₹1 crore will push all ₹30K salaries near 0.
- **When to use:** Neural Networks (inputs expected in `[0,1]`), image pixel scaling (`0-255` → `0-1`), when you need bounded output.

```
from sklearn.preprocessing import MinMaxScaler scaler = MinMaxScaler() # default range=(0,1) scaler = MinMaxScaler(feature_range=(-1, 1)) # custom range X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test)
```

Max-Abs Scaling (MaxAbsScaler)

- **Formula:** $x_{\text{scaled}} = x / |x_{\text{max}}|$
- **Output range:** `[-1, 1]` — divides by the maximum absolute value.
- **When to use:** Sparse data (data with many zeros, like TF-IDF matrices). Does not center the data (zero entries remain zero — doesn't break sparsity).

```
from sklearn.preprocessing import MaxAbsScaler scaler = MaxAbsScaler() X_scaled = scaler.fit_transform(X_train)
```

Robust Scaling (RobustScaler)

- **Formula:** $x_{\text{scaled}} = (x - Q2) / (Q3 - Q1)$ — uses median and IQR instead of mean and std.
- **Output:** No fixed range. Values centered around 0 using median.
- **Robust to outliers:** Yes — median and IQR are not affected by extreme values.
- **When to use:** Data that has significant outliers that you can't or don't want to remove.

```
from sklearn.preprocessing import RobustScaler scaler = RobustScaler() X_scaled = scaler.fit_transform(X_train)
```

Comparison — Which Scaler to Use?

Scaler	Formula	Output Range	Use When	Outlier Sensitive?
StandardScaler	$(x - \text{mean}) / \text{std}$	≈ -3 to $+3$	Default choice, symmetric data	Yes
MinMaxScaler	$(x - \text{min}) / (\text{max} - \text{min})$	<code>[0, 1]</code>	Neural Networks, bounded output needed	Yes
MaxAbsScaler	$x / \text{max} $	<code>[-1, 1]</code>	Sparse data (TF-IDF, text features)	Yes
RobustScaler	$(x - Q2) / (Q3 - Q1)$	No fixed range	Data with outliers you can't remove	No

Normalization vs Standardization:

Normalization usually refers to MinMaxScaler `[0,1]`. Standardization refers to StandardScaler (mean=0, std=1). Both are types of 'feature scaling'. In sklearn, the class is called 'Normalizer' which is something different (it normalizes each ROW to unit norm — used for text/NLP, not the same as MinMaxScaler).

DAY
26

Encoding Categorical Data — Ordinal & Label

youtu.be/w2GglmYHfmM

Why Encoding is Needed

- ML algorithms work with numbers only. Categorical columns (strings like 'Male/Female', 'Low/Medium/High', city names) must be converted to numerical form before training.
- **Two main scenarios:** (1) Ordinal categorical — has a natural order. (2) Nominal categorical — no natural order. The encoding method must match the type.

Ordinal Encoding

- **What:** Assigns a unique integer to each category based on a specified order.
- **When to use:** Categorical features with a meaningful order — Education (High School < UG < PG < PhD), Ratings (Low < Medium < High), Size (S < M < L < XL).
- **Critical:** You must specify the correct order manually. If you let it assign automatically (alphabetically), the model learns wrong relationships (PhD < UG if P > U alphabetically).
- **Works with:** All tree-based models (Decision Tree, Random Forest, XGBoost) and can work with linear models if the ordering is meaningful.

```
from sklearn.preprocessing import OrdinalEncoder oe = OrdinalEncoder(categories=[['Low', 'Medium', 'High']])
# must specify order X_train[['education']] = oe.fit_transform(X_train[['education']])
X_test[['education']] = oe.transform(X_test[['education']]) # Low→0, Medium→1, High→2
```

Label Encoding

- **What:** Converts each unique category to an integer (0, 1, 2, ...) alphabetically by default.
- **When to use: ONLY for the target variable (y).** For example, target = [Cat, Dog, Bird] → [0, 2, 1] (alphabetical).
- **DO NOT use for input features (X):** Label Encoding on nominal features like City (Delhi=0, Mumbai=1, Pune=2) implies a false ordering (Delhi < Mumbai < Pune) which confuses the model.

```
from sklearn.preprocessing import LabelEncoder le = LabelEncoder() y_train = le.fit_transform(y_train) #
e.g., ['Cat', 'Dog'] → [0,1] y_test = le.transform(y_test) print(le.classes_) # ['Bird', 'Cat', 'Dog'] -
alphabetical
```

Ordinal Encoding vs Label Encoding — Key Difference

	Ordinal Encoding	Label Encoding
Applied to	Input features X (with natural order)	Target variable y ONLY
Order	You specify the correct order	Alphabetical (no control)
sklearn class	OrdinalEncoder	LabelEncoder
Input shape	2D array (multiple columns ok)	1D array (single column)
Risk if misused	Using on nominal features implies false order	Using on X features implies false order

Interview trap question:

Q: What's the difference between Label Encoding and Ordinal Encoding? A: Both assign integers to categories, but OrdinalEncoder is for input features X with a known order (you specify it). LabelEncoder is for the target y only and assigns integers alphabetically. Using LabelEncoder on input features X is a mistake — it imposes a false ordering.

DAY
27

One Hot Encoding

youtu.be/U5oCv3JKWKA

What is One Hot Encoding?

- **One Hot Encoding (OHE)** converts a nominal categorical column with N unique categories into N new binary (0/1) columns — one per category.
- **When to use:** Nominal categorical features — no natural order — Gender, Color, City, Blood Group, Brand Name.
- **Why not Label Encoding for nominal?** Label Encoding gives City → Delhi=0, Mumbai=1, Pune=2. This tells the model Mumbai is between Delhi and Pune, which is meaningless. OHE avoids this by treating each category independently.

How it Works

- A column 'Color' with values [Red, Blue, Green] becomes 3 new columns: is_Red, is_Blue, is_Green — each row has exactly one 1 and two 0s.

Color	is_Red	is_Blue	is_Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1

```
from sklearn.preprocessing import OneHotEncoder ohe = OneHotEncoder(sparse_output=False, drop='first') #
drop='first' avoids dummy variable trap X_encoded = ohe.fit_transform(X_train[['city']]) # With pandas
(simpler for EDA): pd.get_dummies(df, columns=['city', 'gender'], drop_first=True)
```

The Dummy Variable Trap

- If a feature has N categories and you create N binary columns, the last column is perfectly predictable from the others (if all others = 0, last = 1). This causes **multicollinearity**.
- **Fix:** Use drop='first' or drop_first=True — creates N-1 columns instead of N. Linear models require this. Tree-based models are unaffected.

High Cardinality Problem

- If a categorical column has 500 unique city names, OHE creates 500 new columns — causes dimensionality explosion and slow training.
- **Solutions for high cardinality:**
 - **Target Encoding:** Replace each category with the mean of the target variable for that category.
 - **Frequency Encoding:** Replace with how often each category appears.
 - **Embedding:** Used in deep learning — learns a dense vector representation.
 - **Bucketing:** Group rare categories into an 'Other' bucket.

OHE practical rule:

Use OHE for nominal columns with low cardinality (< 10-15 unique values). For high cardinality (cities, zip codes, product IDs), use Target Encoding or Frequency Encoding. Always drop one column (drop='first') when using with linear models to avoid multicollinearity.

DAY
28

Column Transformer

youtu.be/5TVj6iEBR4I

The Problem it Solves

- In a real dataset, you typically have mixed column types: some numerical columns (need StandardScaler), some ordinal columns (need OrdinalEncoder), some nominal columns (need OneHotEncoder).
- Without ColumnTransformer, you'd have to manually apply different transformations to different column subsets — complex, error-prone code.
- **ColumnTransformer** applies different transformers to different column subsets in a single, clean, sklearn-compatible step.

How it Works

```
from sklearn.compose import ColumnTransformer from sklearn.preprocessing import StandardScaler,
OneHotEncoder, OrdinalEncoder # Define which transformer applies to which columns ct =
ColumnTransformer(transformers=[ ('scale', StandardScaler(), ['age', 'salary', 'cgpa']), # numerical
('ohe', OneHotEncoder(drop='first'), ['city', 'gender']), # nominal
('ordinal', OrdinalEncoder(categories=[['Low', 'Med', 'High']]), ['rating']), # ordinal ],
remainder='passthrough') # keep any unspecified columns as-is # Fit and transform X_train_transformed =
ct.fit_transform(X_train) X_test_transformed = ct.transform(X_test)
```

Key Parameters

Parameter	Purpose
transformers	List of (name, transformer, columns) tuples — one per transformation group

remainder='drop'	Drops any columns not mentioned in transformers (default)
remainder='passthrough'	Passes unmentioned columns through unchanged
remainder=SomeTransformer	Applies a transformer to all remaining columns

- Output of ColumnTransformer is a **numpy array** — column names are lost. To preserve column names, use `set_output(transform='pandas')` (sklearn >= 1.2) or wrap in a pandas DataFrame manually.

Why use ColumnTransformer instead of applying manually:

ColumnTransformer integrates cleanly into sklearn Pipelines (Day 29). It ensures the same transformations are applied to both train and test consistently, preventing data leakage. It's the industry-standard approach for preprocessing mixed data.

DAY
29

Machine Learning Pipelines

youtu.be/xOccYkgRV4Q

What is an ML Pipeline?

- **Pipeline** chains multiple processing steps into a single sklearn object. Each step's output becomes the next step's input. The last step must be an estimator (model).
- **Why Pipelines?** Without pipelines, you must manually apply each preprocessing step on train and test separately — very easy to forget a step, apply them in the wrong order, or leak test data into training.
- A Pipeline treats the entire workflow — preprocessing + model — as one unit that you can fit, predict, evaluate, and cross-validate in a single call.

Basic Pipeline Structure

```
from sklearn.pipeline import Pipeline from sklearn.preprocessing import StandardScaler from
sklearn.linear_model import LogisticRegression pipe = Pipeline(steps=[ ('scaler', StandardScaler()), #
Step 1: scale features ('model', LogisticRegression()) # Step 2: fit model ]) # Training –
scaler.fit_transform + model.fit happen automatically pipe.fit(X_train, y_train) # Prediction –
scaler.transform + model.predict happen automatically y_pred = pipe.predict(X_test) # Score
print(pipe.score(X_test, y_test))
```

Pipeline with ColumnTransformer — Real World Pattern

```
from sklearn.compose import ColumnTransformer from sklearn.preprocessing import StandardScaler,
OneHotEncoder from sklearn.ensemble import RandomForestClassifier from sklearn.pipeline import Pipeline #
Step 1: ColumnTransformer for preprocessing preprocessor = ColumnTransformer([ ('num', StandardScaler(),
['age', 'salary']), ('cat', OneHotEncoder(drop='first'), ['city', 'gender']), ]) # Step 2: Full pipeline
pipe = Pipeline([ ('preprocessor', preprocessor), ('classifier', RandomForestClassifier()) ])
pipe.fit(X_train, y_train) # preprocesses + trains pipe.predict(X_test) # preprocesses + predicts – no
leakage
```

Advantages of Pipelines

- **No data leakage:** Pipeline ensures preprocessing is always fit on training data only — even inside cross-validation.
- **Cross-validation works correctly:** `cv.cross_val_score(pipe, X, y)` refits the scaler inside each fold — proper evaluation.
- **Deployment-ready:** Save the entire pipeline as one object with pickle/joblib — includes the scaler, encoder, and model together. No need to save/load each component separately.
- **Hyperparameter tuning:** Can tune preprocessing and model parameters together with GridSearchCV.

```
import joblib joblib.dump(pipe, 'model_pipeline.pkl') # save entire pipeline pipe =
joblib.load('model_pipeline.pkl') # load and predict on new data # GridSearchCV on entire pipeline from
sklearn.model_selection import GridSearchCV params = {'classifier__n_estimators': [50, 100],
'classifier__max_depth': [3, 5]} grid = GridSearchCV(pipe, params, cv=5) grid.fit(X_train, y_train)
```

Pipeline is industry standard:

In production ML, you ALWAYS use a Pipeline. It prevents the #1 mistake beginners make — fitting the scaler on train+test combined. A Pipeline + ColumnTransformer combo handles any real-world tabular ML task. Master this pattern completely.

Why Transform Feature Distributions?

- Many ML algorithms (especially linear models) assume features are **roughly normally distributed**. Real-world data often has heavy skew.
- **Right-skewed (positive skew)**: Most values are small, a few very large ones pull the tail right. Common for income, house prices, population. Mean > Median.
- **Goal of transformation**: Reduce skewness → more symmetric distribution → model learns better.

Log Transform — Most Common

- **Formula**: $x_{\text{new}} = \log(x)$ — usually natural log (`np.log`) or log base 10 (`np.log10`).
- **When to use**: Right-skewed data where values span multiple orders of magnitude (salary: ₹10K to ₹1 crore).
- **Requirement**: All values must be POSITIVE (> 0). If data has zeros, use $\log(x + 1) = \text{np.log1p}()$.
- **Effect**: Compresses large values, spreads small values — converts multiplicative relationships to additive.

```
import numpy as np
df['log_salary'] = np.log(df['salary']) # natural log, x > 0
df['log1p_salary'] = np.log1p(df['salary']) # log(x+1), handles x=0
# Check skewness before and after
print('Before:', df['salary'].skew())
print('After:', df['log_salary'].skew()) # should be near 0
```

Square Root Transform

- **Formula**: $x_{\text{new}} = \sqrt{x}$
- **Weaker than log** — use for moderately right-skewed data, or when data has zeros (sqrt handles 0, log doesn't).
- **Requirement**: Values must be ≥ 0 (non-negative).

```
df['sqrt_col'] = np.sqrt(df['count_col']) # handles zeros
import pandas as pd
df['sqrt_col'] = df['count_col'] ** 0.5 # equivalent
```

Reciprocal Transform

- **Formula**: $x_{\text{new}} = 1/x$
- **When to use**: Heavily right-skewed data where you want to flip the relationship (e.g., time taken per unit = $1/\text{speed}$).
- **Requirement**: Values must be non-zero (division by zero error). Also flips the variable direction — higher x becomes lower x_{new} .

Using FunctionTransformer in sklearn

```
from sklearn.preprocessing import FunctionTransformer
import numpy as np

# Log transformer
log_tf = FunctionTransformer(np.log1p, validate=True)

# Sqrt transformer
sqrt_tf = FunctionTransformer(np.sqrt, validate=True)

# Use inside a Pipeline
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ('log_transform', FunctionTransformer(np.log1p)),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

Which Transform to Use?

Skewness	Recommended Transform	Condition
Moderate right skew (0.5 – 1.0)	Square root (sqrt)	Values ≥ 0
High right skew (> 1.0)	Log transform	Values > 0 (or use log1p for ≥ 0)
Very high right skew	Log or Reciprocal	Values > 0
Left skew	Squared or Cube transform	Values can be any
Near-symmetric	No transformation needed	—

Practical tip:

Always plot histograms BEFORE and AFTER transformation to verify the distribution improved. Check `df['col'].skew()` — aim for skewness between -0.5 and +0.5 for near-normal. Never apply log transform blindly to all columns — only apply to columns that are actually skewed.

Important Things From My Side

Key concepts that extend Days 21–30 — important for interviews and real projects.

1. Correlation Doesn't Mean the Feature is Useful for the Model

- A feature might have low correlation with the target but still be useful when combined with other features (interaction effects). Correlation is a linear, univariate measure — it misses non-linear and multivariate relationships.
- Use tree-based feature importance (from Random Forest or XGBoost) alongside correlation — it captures non-linear relationships and interactions.

2. Encoding Choice Depends on the Algorithm

Algorithm	Best Encoding for Categoricals
Linear Regression / Logistic Regression	One Hot Encoding (creates binary columns, no false ordering)
Decision Tree / Random Forest / XGBoost	OrdinalEncoder or even Label Encoding — trees handle orderings well
KNN / SVM / Neural Networks	One Hot Encoding + scaling of resulting binary columns
Gradient Boosting (CatBoost)	Built-in categorical handling — no manual encoding needed

3. The Correct Order of Preprocessing Steps

- Order matters inside a Pipeline. The standard correct order is:
 - **Step 1:** Handle missing values (Imputation) — can't encode or scale NaN values.
 - **Step 2:** Encode categoricals (OHE, OrdinalEncoder) — must happen before scaling.
 - **Step 3:** Feature transformations (Log, Sqrt) — stabilize distribution.
 - **Step 4:** Feature Scaling (StandardScaler / MinMaxScaler) — always the last preprocessing step.
 - **Step 5:** Model — receives fully cleaned, encoded, and scaled data.

4. When to Use Which Scaler — Quick Decision

Situation	Use
Default / symmetric data / no outliers	StandardScaler
Neural Networks or need [0,1] range	MinMaxScaler
Sparse data (text, TF-IDF)	MaxAbsScaler
Data with significant outliers	RobustScaler
Tree-based models (RF, XGBoost, DT)	No scaling needed at all

5. Pandas Profiling Alerts to Watch

- **High Correlation:** Two features correlate > 0.9 — one is likely redundant. Consider dropping one.
- **High Cardinality:** Categorical column with too many unique values — don't use OHE, use target/frequency encoding.
- **Zeros:** A column with many zeros might need special handling (zero-inflated distributions).
- **Constant:** A column with only one unique value — completely useless, drop it.
- **Skewed:** Numerical column with high skewness — apply log or sqrt transform.

End of Day 21–30 Complete Theory Notes · CampusX 100 Days of Machine Learning

Next batch: Day 31–40 covers Power Transformer, Binning, Mixed Variables, Date/Time, and Missing Value Imputation.